📚 جزوه سیستم عامل | فصل ۹: حافظه اصلی

📝 بخش ۵: مبادله (Swapping)

۱. مبادله چیست؟ (Swapping) 🔄

دستورالعمل‌ها و داده‌های یک پردازه برای اجرا باید حتماً در **حافظه اصلی (RAM)** باشند. اما اگر حافظه پر شد چه؟ **مبادله (Swapping)** فرآیندی است که در آن، یک پردازه (یا بخشی از آن) موقتاً از حافظه بیرون کشیده شده و به **دیسک (Backing Store)** منتقل می‌شود و بعداً دوباره به حافظه برگردانده می‌شود.

این کار باعث می‌شود درجه چندبرنامگی (DegreeofMultiprogramming) بالا برود (یعنی برنامه‌های بیشتری نسبت به ظرفیت واقعی رم باز باشند).

الف) مبادله استاندارد (Standard Swapping)

- **روش قدیمی:** کلِ پردازه به‌طور کامل از حافظه به دیسک می‌رود (**Swap Out**) و بعداً کلِ آن برمی‌گردد (Swap In).
- **عیب بزرگ:** انتقال کل پردازه (مثلاً ۱۰۰ مگابایت) زمان زیادی می‌برد. برای همین در سیستم‌عامل‌های مدرن (مثل ویندوز و لینوکس) استفاده نمی‌شود.

ب) مبادله با صفحه‌بندی (Swapping with Paging) ✅

- **روش مدرن:** به‌جای کل پردازه، فقط **صفحات (Pages)** خاصی از آن مبادله می‌شوند.
- **Page Out:** انتقال یک صفحه از رم به دیسک.
- **Page In:** برگرداندن صفحه از دیسک به رم.
- این روش بسیار سریع‌تر و بهینه‌تر است و امروزه وقتی می‌گوییم "Swapping"، منظورمان همین است.

۲. مبادله در موبایل (Mobile Systems) 📱

آیا گوشی‌های ما (اندروید/iOS) هم Swapping دارند؟ **خیر!** (یا خیلی محدود). چرا؟

1. **حافظه فلش (Flash Memory):** گوشی‌ها از هارد دیسک استفاده نمی‌کنند. حافظه فلش محدودیت تعداد نوشتن دارد (WriteEndurance) و با Swapping زیاد زود خراب می‌شود.
2. **سرعت:** سرعت انتقال بین رم و فلش نسبت به رم و CPU پایین است.

راهکار موبایل‌ها چیست؟

- **iOS:** وقتی رم پر شود، از برنامه‌ها می‌خواهد حافظه خود را آزاد کنند (Read-only data مثل کدها را حذف می‌کند). اگر برنامه‌ای گوش نکند، سیستم‌عامل آن را **می‌بندد (Terminate)**.
- **Android:** مشابه iOS عمل می‌کند. قبل از بستن برنامه، وضعیتش (Application State) را ذخیره می‌کند تا وقتی دوباره باز شد، سریع به همان جا برگردد.

📝💻 بخش ۶: معماری اینتل (Intel 32 & 64-bit)

اینتل معماری پیچیده‌ای دارد که ترکیبی از **قطعه‌بندی (Segmentation)** و **صفحه‌بندی (Paging)** است.

۱. معماری IA-32 (سیستم‌های ۳۲ بیتی)

فرآیند ترجمه آدرس در اینتل ۳۲ بیتی دو مرحله‌ای است:

قطعه‌بندی (Segmentation Unit):

- آدرس منطقی (Selector+Offset) را می‌گیرد و به **آدرس خطی (Linear Address)** تبدیل می‌کند.

صفحه‌بندی (Paging Unit):

- آدرس خطی را می‌گیرد و به **آدرس فیزیکی (Physical Address)** تبدیل می‌کند.

ساختار صفحه‌بندی در IA-32

از **صفحه‌بندی دو سطحی** استفاده می‌کند:

1. **Page Directory (دایرکتوری صفحه):** ۱۰ بیت اول آدرس.
2. **Page Table (جدول صفحه):** ۱۰ بیت دوم.
3. **Offset:** ۱۲ بیت آخر (برای صفحات ۴ کیلوبایتی).

یک ثبات خاص به نام **CR3** به دایرکتوری صفحه اشاره می‌کند.

تکنیک PAE (Physical Address Extension)

- چون سیستم‌های ۳۲ بیتی فقط ۴ گیگ رم را اساپورت می‌کنند (${32}^2$)، اینتل تکنیک **PAE** را معرفی کرد.
- با PAE، آدرس فیزیکی به ۳۶ بیت افزایش می‌یابد (پشتیبانی تا ۶۴ گیگابایت رم).
- ساختار صفحه‌بندی به **۳ سطح** تغییر می‌کند.

۲. معماری 86-64x (سیستم‌های ۶۴ بیتی)

- فضای آدرس ۶۴ بیتی بسیار عظیم است (${64}^2$)، اما فعلاً فقط از **۴۸ بیت** آن استفاده می‌شود.
- از **صفحه‌بندی ۴ سطحی** استفاده می‌کند تا بتواند این فضای بزرگ را مدیریت کند.
- برخلاف IA-32، اینجا دیگر قطعه‌بندی تقریباً حذف شده و تمرکز روی صفحه‌بندی خالص است.

📝 بخش ۷: معماری ARMv8 (مخصوص موبایل) 🤖

پردازنده‌های ARM (که در اکثر گوشی‌های موبایل و تبلت‌ها مثل آیفون و سامسونگ استفاده می‌شوند) ساختار متفاوتی دارند.

ویژگی‌های کلیدی ARMv8:

دانه‌های ترجمه (Translation Granules):

- برخلاف اینتل که معمولاً روی ۴KB قفل است، ARM سه اندازه صفحه متفاوت را پشتیبانی می‌کند: **۴KB، ۱۶KB و ۶۴KB**.

سطوح صفحه‌بندی:

- بسته به اندازه صفحه، می‌تواند ۳ یا ۴ سطح جدول صفحه داشته باشد.

دو سطح TLB:

- **Micro TLB:** خیلی سریع و کوچک (یکی برای دستورات، یکی برای داده‌ها).
- **Main TLB:** بزرگتر و کندتر. اگر در Micro پیدا نشد، اینجا را می‌گردد.

📊 جدول مقایسه نهایی (جمع‌بندی فصل ۹)

ویژگی	اینتل (IA-32)	اینتل (86-64x)	ARMv8 (موبایل)
تبدیل آدرس	قطعه‌بندی + صفحه‌بندی	صفحه‌بندی ۴ سطحی	صفحه‌بندی ۳ یا ۴ سطحی
اندازه صفحه	۴KB یا ۴MB	۴KB, ۲MB, ۱GB	۴KB, ۱۶KB, ۶۴KB
ویژگی خاص	**PAE** (برای رم >۴ گیگ)	فضای آدرس ۴۸ بیتی	**Micro TLB** (دوسطحی)
مبادله (Swap)	دارد (روی هارد/SSD)	دارد	**ندارد** (برنامه‌ها بسته می‌شوند)